

MANTENIMIENTO DEL EPN EVENT MANAGER

Comparativa antes vs. después • carpeta epn-event-manager

Objetivo

Corregir el Hub central que recibe eventos CREATE, UPDATE, DELETE y QUERY desde los CRUDs. La sustentación se enfoca en diagnóstico, intervención, justificación y prueba de vida.

Correctivo

Bug DELETE y campo description

Adaptativo

Fechas ISO, CORS y ValidationPipe

Perfectivo

Stats + orden

Preventivo

DTO + pipes, Health real

Arquitectura revisada del Hub

epon-event-manager
NestJS + TypeORM + SQLite
Puerto 3000

Recibe POST /events

4 tablas separadas
create_events
update_events
delete_events
query_events

Diagnóstico general: el diseño separa los eventos por tipo. Por eso era fácil que una tabla quedara incompleta, que las fechas no coincidieran o que las estadísticas omitieran información.

Resultado esperado: que el Hub guarde CREATE, UPDATE, DELETE y QUERY; que los liste ordenados; que calcule métricas reales; y que rechace entradas inválidas.

DIAGNÓSTICO GENERAL DEL HUB

Problemas encontrados

- DELETE se creaba pero no se guardaba.
- delete_events no tenía description.
- Fechas con formatos inconsistentes.
- CORS solo permitía un cliente.
- Validaciones mezcladas o insuficientes.
- Estadísticas y ordenamiento incompletos.

Resultado esperado

- Todos los eventos se persisten en SQLite.
- El Hub acepta los cuatro CRUDs.
- Entradas inválidas se rechazan antes del service.
- /events muestra datos ordenados.
- /events/stats entrega métricas útiles.
- /health comprueba la conexión real.



Correctivo C-1: DELETE no se guardaba

Diagnóstico: el objeto DELETE se creaba en memoria, pero nunca se ejecutaba `save()`. La API respondía ok aunque no se persistía en SQLite.

ANTES — `events.service.ts`

```
        if (action === 'DELETE') {  
this.deleteRepo.create({  
  source: dto.source,  
  entity: dto.entity,  
  action: dto.action,  
  title: dto.title,  
  payload: payloadStr,  
  createdAt: localDate,  
});  
return { ok: true };  
}
```

DESPUÉS — `events.service.ts`

```
        if (action === 'DELETE') {  
const ev = this.deleteRepo.create({  
  ...baseData,  
  createdAt: eventDate,  
});  
await this.deleteRepo.save(ev);  
return { ok: true, action };  
}
```

Justificación: es correctivo porque repara una falla funcional existente: antes se perdían eventos DELETE; ahora se guardan correctamente.



Correctivo C-2: delete_events no tenía description

Diagnóstico: DeleteEventEntity no tenía la columna description, aunque las otras entidades sí. Se perdía información del evento DELETE.

ANTES — delete-event.entity.ts

```
title: string;
@Column({ nullable: true })

// No existía description

@Column({ type: 'text', nullable: true })
payload: string;
```

DESPUÉS — delete-event.entity.ts

```
title: string;
@Column({ nullable: true })

description: string;

@Column({ type: 'text', nullable: true })
payload: string;
```

Impacto: DELETE queda consistente con CREATE, UPDATE y QUERY. El historial conserva también la descripción.

JUSTIFICACIÓN DEL CORRECTIVO

¿Por qué es correctivo?

Porque corrige una falla real del sistema: los eventos DELETE no se persistían. El usuario podía eliminar un registro en un CRUD, pero el Hub no conservaba el historial de esa acción.

Impacto: ahora CREATE, UPDATE, DELETE y QUERY quedan registrados, por lo que el Event Manager mantiene trazabilidad completa.

2. ADAPTATIVO: CORS PARA VARIOS CRUDS

ANTES: main.ts

```
app.enableCors({  
  origin: [  
    "http://localhost:4000",  
    "http://127.0.0.1:4000"  
  ],  
  methods: ["GET", "POST", "PUT", "DELETE",  
    "OPTIONS"],  
  allowedHeaders: ["Content-Type"],  
});  
// Solo funcionaba un cliente.
```

DESPUÉS: main.ts

```
app.enableCors({  
  origin: [  
    "http://localhost:4000", "http://127.0.0.1:4000",  
    "http://localhost:4001", "http://127.0.0.1:4001",  
    "http://localhost:4002", "http://127.0.0.1:4002",  
    "http://localhost:4003", "http://127.0.0.1:4003"  
  ],  
  methods: ["GET", "POST", "PUT", "DELETE",  
    "OPTIONS"],  
  allowedHeaders: ["Content-Type"],  
});
```




2. ADAPTATIVO: FECHAS ISO 8601

ANTES `events.service.ts`

```
event_date: new Date().toLocaleString()  
recorded_at: new Date().toLocaleString()
```

Problema:

El formato cambia según el equipo, idioma o región.
Dificulta ordenar eventos.

DESPUÉS `events.service.ts`

```
const now = new Date().toISOString();
```

```
event_date: now  
recorded_at: now
```

Ventaja:

Formato estándar ISO 8601.

Ejemplo: 2026-05-21T02:37:10.519Z

Diagnóstico: `toLocaleString()` depende del idioma y configuración del sistema. Esto dificulta parsear, ordenar e integrar clientes externos.

Justificación: es adaptativo porque el sistema se ajusta a un estándar externo universal de fechas para interoperar mejor con otros servicios.



3. PERFECTIVO: ESTADÍSTICAS Y ORDENAMIENTO

ANTES `events.service.ts`

```
// Estadísticas parciales
// No siempre incluían todas las acciones.

return {
  create: totalCreate,
  update: totalUpdate
};

// Ordenamiento inconsistente por fecha.
```

DESPUÉS `events.service.ts`

```
return {
  total,
  byAction: {
    CREATE: totalCreate,
    UPDATE: totalUpdate,
    DELETE: totalDelete,
    QUERY: totalQuery
  },
  bySource,
  byEntity
};

// /events ordena por _eventDate descendente.
```

Diagnóstico: `getStats()` omitía `query_events`. El total era menor al real y el reporte no representaba todo el uso del sistema.

Justificación: es perfectivo porque mejora una funcionalidad existente: las métricas ahora dan más valor y son completas.



Perfectivo P-2: historial ordenado y normalizado

Diagnóstico: se mezclaban fechas de columnas distintas y se ordenaban como texto. El historial podía quedar en orden incorrecto.

ANTES — orden por string

```
merged.sort((a, b) => {  
  const ta = ra.recorded_at ?? ra.timestamp  
    ?? ra.createdAt ?? ra.event_date ?? "";  
  const tb = rb.recorded_at ?? rb.timestamp  
    ?? rb.createdAt ?? rb.event_date ?? "";  
  return ta.localeCompare(tb);  
});
```

DESPUÉS — Date.parse()

```
return merged.sort((a, b) => {  
  const ta = Date.parse(String(a._eventDate ?? "")) ||  
    0;  
  const tb = Date.parse(String(b._eventDate ?? "")) ||  
    0;  
  return tb - ta;  
});
```

Además se usa `normalizeEvents()`: cada registro recibe `_table` y `_eventDate` para unificar `create/update/delete/query`.



Preventivo Pv-1: health check real

Diagnóstico: /health siempre respondía ok aunque la base de datos pudiera estar fallando. Eso ocultaba problemas futuros.

ANTES — health.controller.ts

@Get()

```
check() {  
  return {  
    status: 'ok',  
    timestamp: new Date().toLocaleString(),  
  };  
}
```

DESPUÉS — health.controller.ts

@Get()

```
async check() {  
  const databaseConnected = this.dataSource.isInitialized;  
  if (databaseConnected) {  
    await this.dataSource.query('SELECT 1');  
  }  
  return {  
    status: databaseConnected ? 'ok' : 'error',  
    database: databaseConnected ? 'connected' : 'disconnected',  
    timestamp: new Date().toISOString(),  
  };  
}
```

Justificación: es preventivo porque detecta fallos de BD antes de que los CRUDs sigan enviando eventos a ciegas.

4. PREVENTIVO: DTO + VALIDATIONPIPE

ANTES create-event.dto.ts

```
// Validaciones insuficientes o mezcladas  
// en el service.
```

```
if (!payload.source) throw new Error(...);  
if (!payload.action) throw new Error(...);
```

```
// El service recibía datos sucios.
```

DESPUÉS: create-event.dto.ts

```
@IsString()  
@IsNotEmpty()  
@MaxLength(80)  
source: string;
```

```
@IsIn(["CREATE", "UPDATE", "DELETE", "QUERY"])  
action: EventAction;
```

```
@IsOptional()  
@MaxLength(500)  
description?: string;
```



Preventivo Pv-2: DTO con validaciones

Diagnóstico: el DTO original aceptaba campos vacíos, acciones no permitidas, payloads no controlados y textos demasiado largos.

ANTES — create-event.dto.ts

```
export class CreateEventDto {  
  source: string;  
  entity: string;  
  action: string;  
  title: string;  
  description: string;  
  payload: any;  
}
```

DESPUÉS — create-event.dto.ts

```
    @IsString() @IsNotEmpty() @MaxLength(60)  
    source!: string;  
  
    @IsString() @IsNotEmpty()  
    @IsIn(['CREATE', 'UPDATE', 'DELETE', 'QUERY'])  
    action!: string;  
  
    @IsString() @IsNotEmpty() @MaxLength(120)  
    title!: string;  
  
    @IsOptional() @IsObject()  
    payload?: Record<string, unknown>;
```

Impacto: si llega action = BORRAR, source vacío o propiedades extra, el Hub puede rechazar la solicitud con 400 Bad Request.

PREVENTIVO: SEPARACIÓN DE RESPONSABILIDADES

DTO

Valida estructura, límites, campos obligatorios y valores permitidos.

Controller

Recibe la petición y delega al service.

Service

Procesa datos ya validados y decide en qué tabla guardar.

Repository

Persiste en SQLite.

PRUEBA DE VIDA DEL HUB

Demostración sugerida

1. Levantar epn-event-manager en <http://localhost:3000>.
2. Abrir /health y verificar database: connected.
3. Ejecutar los CRUDs en 4000, 4001, 4002 y 4003.
4. Crear, actualizar, consultar y eliminar registros.
5. Abrir /events y buscar acciones CREATE, UPDATE, DELETE y QUERY.
6. Abrir /events/stats para ver métricas consolidadas.

MEJORAS FUTURAS OPCIONALES

Opciones técnicas

- Mover CORS a variables .env.
- Crear filtros:
/events?source=...&action=...
- Agregar paginación: page y limit.
- Implementar pruebas automáticas.
- Mejorar logs y mensajes de error.

Mejora arquitectónica

- Reemplazar las 4 tablas por una tabla única events con campo action.
- Mantener DTOs por acción si las reglas crecen.
- Estandarizar un único campo de fecha para todas las tablas.